

LAYER 1 SPECIFICATION · HARDWARE & ENVIRONMENT ATTESTATION

Layer 1 — Device & Environment Integrity Attestation

Note (Oct 2026): this page covers the Android prototype. The desktop/Linux Layer 1 engine (`host_integrity.py` : V4L2/sysfs camera attestation plus the active sensor challenge) is described in [ARCHITECTURE.md](#), Section 4.1–4.2.

Branch: `device-integrity-attestation` Part of: **AI-Deepfake-Detection-Innovative-Design-Project**

Overview

This module implements **Layer 1** of the project's five-layer detection pipeline. It runs before any deepfake-specific analysis (PRNU sensor-noise profiling, temporal/frequency analysis, or rPPG liveness) and answers one question: *can this device and camera source be trusted at all?*

Digital injection attacks — where an attacker feeds a pre-rendered deepfake video into an app's camera pipeline using virtual camera software or SDK hooking — almost always depend on a compromised or virtualized environment to do so. This layer targets that dependency directly, rather than the video content itself.

What it checks

Sub-check	File	What it detects
Root / Jailbreak Detection	<code>RootDetector.kt</code>	<code>su</code> binaries, root management apps (Magisk, SuperSU, etc.), writable system partitions, test-keys build signature
Emulator Detection	<code>EmulatorDetector.kt</code>	Build fingerprint/model/hardware markers (goldfish, ranchu, sdk_gphone), QEMU pipe files, missing baseline sensors
Virtual Camera Detection	<code>VirtualCameraDetector.kt</code>	Camera enumeration anomalies, known virtual-camera signatures, suspicious hardware support levels

Each detector returns a standardized `DetectionResult(flagged, confidence, signals)` so results can be combined uniformly downstream.

How the verdict is decided

`DeviceIntegrityChecker.kt` runs all three detectors and applies a simple gating rule:

- **Root or Emulator flagged** → **BLOCK**. Either one compromises the trustworthiness of every later layer, so the session is terminated immediately without running the expensive PRNU/FFT/rPPG analysis.
- **Only Virtual Camera flagged** → **FLAG_FOR_REVIEW**. On Android this signal is treated as corroborating rather than definitive (see docstring in `VirtualCameraDetector.kt` for why), so it doesn't independently block.
- **Nothing flagged** → **PASS**. Session proceeds to Layer 2.

This is the "gated inference fusion" strategy from the project's overall architecture — Layer 1 exists specifically to make early termination possible before costly downstream computation.

Design notes / limitations (for report & panel questions)

- Manual heuristic checks (su paths, build tags, etc.) are individually bypassable by an attacker with runtime hooking (e.g. Frida, Magisk Hide). Combining multiple independent signals raises the bar; a production system would additionally use Google's **Play Integrity API** for hardware-backed attestation as an authoritative server-side check.
- False positives are possible on legitimate rooted devices used by power users, not attackers — this is a known trade-off and worth discussing as a limitation in the report.
- Virtual camera detection is comparatively weak on Android versus desktop/web (where tools like OBS Virtual Camera are far more common and easier to fingerprint by device name). If the project scope extends to a web/desktop KYC client, the same signature-matching approach should be re-applied against `MediaDevices.enumerateDevices()` results.

Measured metric (for Review 2)

`Layer1Verdict.processingTimeMs` reports end-to-end detection latency. Target: **sub-50ms**, so this layer does not become a bottleneck relative to the heavier neural-network-based layers.

How to run the demo

1. Open the project in Android Studio.
2. Run on a physical device to see a clean `PASS` verdict.
3. Run on an emulator (e.g. default Android Studio AVD) to see the `EmulatorDetector` correctly flag it — this is an easy way to demonstrate the module actually working during a review.

File structure

```
app/src/main/java/com/idp/deviceintegrity/  
├── DetectionResult.kt      # shared result data class  
├── RootDetector.kt         # Layer 1a  
├── EmulatorDetector.kt     # Layer 1b  
├── VirtualCameraDetector.kt # Layer 1c  
├── DeviceIntegrityChecker.kt # orchestrator + gating logic  
└── MainActivity.kt        # demo UI
```